

Java Object Class

The Root of All Classes in Java | Common Methods | Why It Exists | Override for Meaningful Behavior

Every Class
Inherits Object
(Implicitly)



1 What is Object?

- ▶ **Object** is a special class in java defined in **java.lang**.
- ▶ Every class automatically inherit this class whether we say it or not.

We write like...

```
class Employee{
    int id;
    double salary;
    ....
    ....
}
```

Java compiler convert it as...

```
class Employee extends Object{
    int id;
    double salary;
    ....
    ....
}
```

2 Why Java has provided this class?

- ▶ Provides common behavior to all classes.
- ▶ Defines methods that are useful for every object (e.g. toString, equals, hashCode, etc).
- ▶ Supports core functionality like cloning, garbage collection, thread communication, getting class information etc.
- ▶ Allows every object to be treated uniformly.

3 Methods defined in Object class

Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned.
boolean equals(Object object)	Determines whether one object is equal to another.
void finalize()	Called before an unused object is recycled.
Class<?> getClass()	Obtains the class of an object at run time.
int hashCode()	Returns the hash code associated with the invoking object.
void notify()	Resumes execution of a thread waiting on the invoking object.
void notifyAll()	Resumes execution of all threads waiting on the invoking object.
String toString()	Returns a string that describes the object.
void wait()	Waits on another thread of execution.
void wait(long milliseconds)	
void wait(long milliseconds, int nanoseconds)	

4 toString()

- ▶ If we do not override **toString()** method of Object class it print Object Identification number by default.
- ▶ We can override it to print some useful information.

Employee class

```
class Employee{
    private int id;
    private double salary;

    public Employee(int id, double salary) {
        this.id = id;
        this.salary = salary;
    }

    ....

    Employee e=new Employee(22, 33333.5);
    System.out.println(e);
    ....
}
```

O/P

com.Employee@adbf1

Java simply print object
identification number
not so useful message
for client

Overriding toString()

```
@Override
public String toString() {
    return "Employee [id=" + id + ",
        salary=" + salary + "];"
}
```

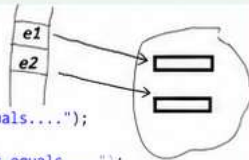
Output (after override)

Employee [id=22, salary=33333.5]

5 equals()

```
Employee e1=new Employee(22, 33333.5);
Employee e2=new Employee(22, 33333.5);
```

```
if(e1==e2)
    System.out.println("two employees are equals....");
else
    System.out.println("two employees are not equals....");
```



Output would be: two employees are not equals....

Problem is that using **==** java compare object id of two object and that can never be equals, so we are getting meaningless result.

6 Overriding equals()

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Employee other = (Employee) obj;
    if (id != other.id)
        return false;
    if (Double.doubleToLongBits(salary) !=
        Double.doubleToLongBits(other.salary))
        return false;
    return true;
}
```

What this does?

- ▶ Checks for same reference (this == obj)
- ▶ Checks for null
- ▶ Checks for same class
- ▶ Compares field values (id, salary)
- ▶ Returns true if all match

7 hashCode()

- ▶ Whenever you override equals() for any type don't forget to override hashCode() method.
- ▶ hashCode() make DS efficient. What hashCode does HashCode divide data into buckets.
- ▶ Equals search data from that bucket.

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + id;
    long temp;
    temp = Double.doubleToLongBits(salary);
    result = prime * result + (int) (temp ^ (temp >> 32));
    return result;
}
```



Key Takeaway

Object is the root class of all Java classes. Override equals(), hashCode() and toString() to provide meaningful behavior for your own classes.

Better Objects
Better Applications!



Java Collections Framework

Data Structures | Algorithms | Common Implementations | Performance (Big O)

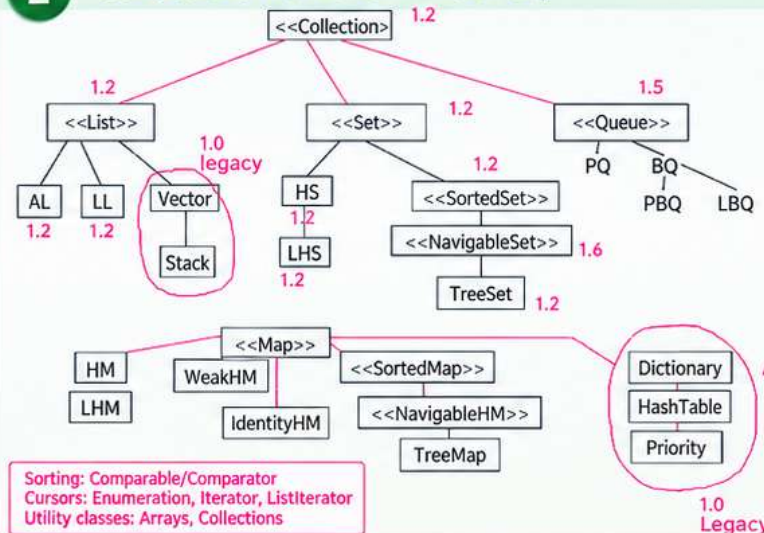
Right Data Structure
Right Performance
Better Applications

Java™

1 What is Java Collections API?

- A unified architecture to store and manipulate groups of objects.
- Part of **java.util** package.
- Provides interfaces, implementations (classes), and algorithms.
- Reduces programming effort and improves performance.
- Built on top of standard data structures.

2 Collections Framework Hierarchy



3 Four type of collections

Collections come in four basic flavors:

- **Lists** Lists of things (classes that implement List).
- **Sets** Unique things (classes that implement Set).
- **Maps** Things with a unique ID (classes that implement Map).
- **Queues** Things arranged by the order in which they are to be processed.

4 Data Structure vs Algorithm

Data Structure

- A way to organize and store data in memory.
- Examples: Array, Linked List, Stack, Queue, Hash Table, Tree, Graph.
- Used by Collections Framework internally.



Algorithm

- A step-by-step procedure to solve a problem or perform an operation on data.
- Examples: Searching, Sorting, Insertion, Deletion, Traversal.
- Collections provides ready to use algorithms (e.g. sort, search).



5 Common Data Structures (Used in Collections)

Data Structure	Brief Description	Used in (Java Collections)
Array	Fixed size, contiguous memory	ArrayList (internally), Arrays
Linked List	Nodes connected via links	LinkedList
Stack	LIFO (Last In First Out)	Stack (legacy)
Queue	FIFO (First In First Out)	Queue, PriorityQueue, BlockingQueue
Hash Table	Key-value with hashing	HashMap, Hashtable, LinkedHashMap, IdentityHashMap, WeakHashMap
Tree	Sorted data (Red-Black Tree)	TreeSet, TreeMap
Graph	Nodes and edges	(Not in core Collections, but used in many applications)

6 Common Algorithms (Used in Collections)

Algorithm	Purpose / Used for
Sorting	Order elements (Collections.sort())
Searching	Find an element (Collections.binarySearch())
Insertion	Add elements
Deletion	Remove elements
Traversal	Iterate through elements (Iterator, ListIterator, Enumeration)
Comparison	Compare elements (Comparable/Comparator)
Hashing	Compute hash code (hashCode())



Collections also provides utility classes like Arrays and Collections for common algorithms.

7 Performance (Big O) of Common Java Collection Classes

Class / Interface	Add	Remove	Search (contains)	Notes
ArrayList	O(1)*	O(n)	O(n)	Fast random access, slow middle insert/delete
LinkedList	O(1)	O(1)	O(n)	Good for frequent insert/delete
Vector (legacy)	O(1)*	O(n)	O(n)	Synchronized, legacy class
Stack (legacy)	O(1)	O(1)	O(n)	LIFO, synchronized (legacy)
HashSet	O(1)	O(1)	O(1)	No duplicates, unordered
LinkedHashSet	O(1)	O(1)	O(1)	Maintains insertion order
TreeSet	O(log n)	O(log n)	O(log n)	Sorted order (Red-Black Tree)
HashMap	O(1)	O(1)	O(1)	Key-value, no order
LinkedHashMap	O(1)	O(1)	O(1)	Maintains insertion order
TreeMap	O(log n)	O(log n)	O(log n)	Sorted by keys
PriorityQueue	O(log n)	O(log n)	O(n)	Elements ordered by priority

* Amortized time complexity



Key Takeaways

- Java Collections API provides List, Set, Map and Queue to work with groups of objects.
- Choose the right collection based on data structure and performance needs.
- Use algorithms (sorting, searching, etc.) provided by Collections for common operations.
- Understand Big O to write efficient and scalable applications.

Right Collection.
Right Algorithm.
Better Performance.



ArrayList vs LinkedList

Two List Implementations in Java Collections | Internal Structure | Performance | When to Use

Choose the Right List
for Better Performance



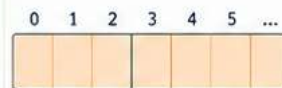
1 Key Points from PPT

Java implements ArrayList as array internally

- Hence good to provide starting size
- i.e. `List<String> s = new ArrayList<String>(20);` is better than `List<String> s = new ArrayList<String>();`
- Removing element from starting of arraylist is very slow
- `list.remove(0);`
- if u remove first element, Java internally copy all the element (shift by one)
- Adding element at middle in ArrayList is very inefficient...

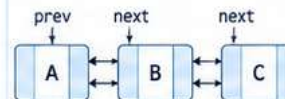
2 Internal Implementation

ArrayList: aka growable array



Implemented using a dynamic array (grows as needed)

LinkedList: doubly linked list



Implemented as a doubly linked list (each element holds reference to previous and next node)

3 Sample Code – Performance Comparison

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

public class App {
    public static void main(String[] args) {
        List<Integer> arrayList = new ArrayList<Integer>();
        List<Integer> linkedList = new LinkedList<Integer>();

        doTimings("ArrayList", arrayList);
        doTimings("LinkedList", linkedList);
    }
}
```

```
private static void doTimings(String type, List<Integer> list) {
    for(int i=0; i<1E5; i++)
        list.add(i);
    long start = System.currentTimeMillis();

    // Add items at end of list
    for(int i=0; i<1E5; i++) {
        list.add(i);
    }

    // Add items elsewhere in list
    for(int i=0; i<1E5; i++) {
        list.add(0, i);
    }

    long end = System.currentTimeMillis();

    System.out.println("Time taken: " + (end - start) +
        " ms for " + type);
}
```

Time taken: 7546 ms for ArrayList
Time taken: 76 ms for LinkedList

4 Common Operations – Performance (Big O)

Operation	ArrayList	LinkedList	Notes
Add at end	$O(1)^*$	$O(1)$	ArrayList may resize (amortized)
Add at beginning	$O(n)$	$O(1)$	ArrayList shifts elements
Add at middle	$O(n)$	$O(1)$	LinkedList just updates links
Get element (by index)	$O(1)$	$O(n)$	ArrayList direct access, LinkedList traversal
Set element (by index)	$O(1)$	$O(n)$	LinkedList traversal
Remove at end	$O(1)$	$O(1)$	
Remove at beginning	$O(n)$	$O(1)$	ArrayList shifts elements
Remove at middle	$O(n)$	$O(1)$	ArrayList shifts elements
Search (contains)	$O(n)$	$O(n)$	

5 ArrayList Example

```
List<String> list = new ArrayList<String>();
...
list.size();
list.contains("raj");

test.remove("hi");

Collections.sort(list);
```

ArrayList provides fast random access but is inefficient for insertions/deletions at the beginning or middle.

6 Useful Utility Methods (from PPT)

```
Collections.sort(list, Collections.reverseOrder());

Collections.addAll(list2, list1);
// Add all elements from list1 to end of list2

Collections.frequency(list2, "foo");
// print frequency of "foo" in the list2 collection

boolean flag = Collections.disjoint(list1, list2);
// return "true" if nothing is common in list1 and list2

Sorting with the Arrays Class

Arrays.sort(arrayToSort)
Arrays.sort(arrayToSort, Comparator)
```

7 When to Use?

Use ArrayList when:

- You need fast random access (get/set by index)
- Mostly adding/removing at the end
- You can provide an initial capacity (`new ArrayList<>(size)`)

Use LinkedList when:

- Frequent insertions/deletions at the beginning or middle
- You don't need fast random access
- Implementing Queue, Deque, etc.



Key Takeaway

ArrayList is backed by a growable array – fast access, slow middle/beginning updates. LinkedList is a doubly linked list – efficient insertions/deletions, but slow random access. Choose the right implementation based on your use case and performance needs.

Java Comparable vs Comparator

Two ways to tell Java how to sort objects | Natural Ordering vs Custom Ordering

Same Data
Different Sorting
Same Power



1 Why do we need it?

- We need to teach Java how to sort user defined objects.
- Comparable and Comparator interface help us to tell Java how to sort user defined objects.
- Used with Collections.sort(list); or Arrays.sort(array).

2 Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Purpose	Natural sort	Secondary sorts
Number of sort sequences	Only one sort sequence is possible	As many as you want
Change in class design	Need to change the design of the class	Do not need to change design of the class
Method to override	public int compareTo(Employee o)	public int compare(Employee o1, Employee o2)

3 ArrayList of user defined object

```
class Employee{
    int id;
    float salary;
    //getter setter
    //const
    //toString
}
```

```
List<Employee>list = new ArrayList<Employee>();
list.add(new Employee(121, "rama"));
list.add(new Employee(121, "rama"));
list.add(new Employee(121, "rama"));
```

```
System.out.println(list);
```

```
Collections.sort(list);
```

How java can decide how to sort?

4 Implementing Comparable (Natural Ordering)

```
class Employee implements Comparable<Employee>{
    private int id;
    private double salary;
    .....
    .....

    @Override
    public int compareTo(Employee o) {
        // TODO Auto-generated method stub
        Integer id1 = this.getId();
        Integer id2 = o.getId();
        return id1.compareTo(id2);
    }
}
```

Key Point

- Define natural ordering inside the class.
- Usually used for primary/default sorting (e.g. by id).

5 Using Comparator (Custom Ordering)

```
class SalarySorter implements Comparator<Employee>{

    @Override
    public int compare(Employee o1, Employee o2) {
        // TODO Auto-generated method stub
        Double sal1 = o1.getSalary();
        Double sal2 = o2.getSalary();
        return sal1.compareTo(sal2);
    }
}
```

Key Point

- Do not need to change the Employee class.
- Can define multiple sorting logic (e.g. by salary, by name).
- Pass comparator to sorting methods.
- Commonly used for secondary or custom sorts.

6 Example: Sorting a List

```
List<Employee> list = new ArrayList<Employee>();
list.add(new Employee(121, "rama"));
list.add(new Employee(102, "sita"));
list.add(new Employee(150, "raj"));
```

```
// Natural ordering (uses Comparable - by id)
Collections.sort(list);
```

```
// Custom ordering (uses Comparator - by salary)
Collections.sort(list, new SalarySorter());
```

Output

- First sort (Comparable): elements sorted by id.
- Second sort (Comparator): elements sorted by salary.



Key Takeaways

- Use Comparable when you have a natural/default ordering for the class.
- Use Comparator when you need multiple or custom sorting logic without modifying the class.
- Both are widely used with Collections.sort() and Arrays.sort().

Comparable for natural sort
Comparator for flexible sort!

Java HashMap

Key-Value Storage | Internal Structure | Examples | Iteration | Use Cases

Fast Access
Flexible Usage
Powerful Collection



1 What is HashMap?

- HashMap is a part of **java.util** package.
- Stores data in key-value pairs.
- Keys are unique (no duplicate keys).
- Allows null key and multiple null values.
- Not synchronized (not thread-safe).
- Does not maintain any order.
- Provides constant time performance $O(1)$ for basic operations (get, put, remove) on average.



Best when you need fast access using keys and don't care about the order.

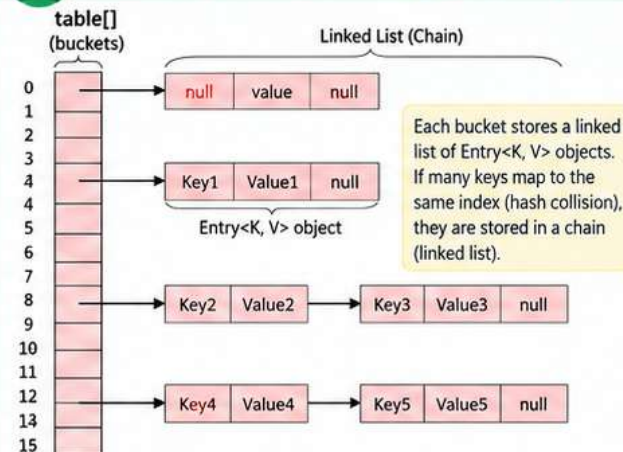
2 HashMap Internal Structure

- HashMap stores the data in the form of key-value pairs.
- Each key-value pair is stored in an object of **Entry<K, V>** class.

```
static class Entry<K,V> implements Map.Entry<K,V>
{
    final K key;
    V value;
    Entry<K,V> next;
    int hash;
}
```

- **key** : It stores the key of an element and is final.
- **value** : It holds the value of an element.
- **next** : It holds the pointer to next key-value pair. This makes the key-value pairs stored as a linked list.
- **hash** : It holds the hashCode of the key.

3 HashMap Internal Working (Diagram)



4 How it Works?

1. **Compute hash code**
`int hash = key.hashCode();`

2. **Find index in table**
`int index = (n - 1) & hash;`
(n = table length, power of 2)

3. **Check the bucket**
If **empty** → add new entry
If keys match → update value
If collision → traverse linked list (to find key or add new entry)

5 Example: Put and Get

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();

        map.put(5, "Five");
        map.put(8, "Eight");
        map.put(6, "Six");
        map.put(4, "Four");
        map.put(2, "Two");

        String value = map.get(6); // Six
        System.out.println("Value for key 6: " + value);
    }
}
```

Output:

Value for key 6: Six

6 Iterating a HashMap

```
import java.util.*;

public class IterateHashMap {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(5, "Five");
        map.put(8, "Eight");
        map.put(6, "Six");
        map.put(4, "Four");
        map.put(2, "Two");

        // 1. Using keySet()
        for (Integer key : map.keySet()) {
            System.out.println(key + " -> " + map.get(key));
        }

        // 2. Using entrySet() (recommended)
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }

        // 3. Using forEach (Java 8+)
        map.forEach((key, value) ->
            System.out.println(key + " -> " + value));
    }
}
```

7 Common Operations & Time Complexity

Operation	Average Time	Worst Case	Description
put(key, value)	$O(1)$	$O(n)$	Add or update value for a key
get(key)	$O(1)$	$O(n)$	Get value by key
remove(key)	$O(1)$	$O(n)$	Remove entry by key
containsKey(key)	$O(1)$	$O(n)$	Check if key exists
containsValue(value)	$O(n)$	$O(n)$	Check if value exists
size()	$O(1)$	$O(1)$	Number of entries
isEmpty()	$O(1)$	$O(1)$	Check if map is empty
keySet()	$O(1)$	$O(1)$	Set of keys
values()	$O(1)$	$O(1)$	Collection of values
entrySet()	$O(1)$	$O(1)$	Set of key-value pairs



- Average time is $O(1)$ because of hashing.
- Worst case is $O(n)$ when many keys fall into the same bucket (hash collisions).
- Use good `hashCode()` and `equals()` implementations for custom keys.

8 Key Takeaways

- ✓ HashMap stores key-value pairs.
- ✓ Keys are unique, values can be duplicate.
- ✓ Allows one null key and multiple null values.
- ✓ Does not maintain any order.
- ✓ Provides $O(1)$ performance for basic operations (on average).
- ✓ Use `keySet()`, `entrySet()` or `forEach()` to iterate.
- ✓ Best choice for fast key-based access.

“HashMap is like a dictionary – find the value quickly using a unique key, without worrying about the order.”



Java Set

Unique Elements | No Duplicates | Different Implementations | Choose as per Need

Same Elements
Only Once!
That's a Set!

1 What is Set?

- Set is a part of **java.util** package.
- It stores **unique elements** (no duplicates).
- It does not maintain insertion order (except **LinkedHashSet**).
- Allows at most **one null element** (implementation dependent).
- Useful when you want to represent a group of **distinct objects**.



Think of a set like a collection of books – the same book will not appear twice.

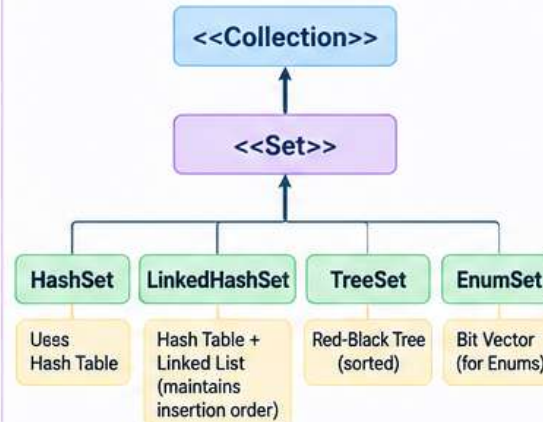
2 Real-World Analogy

Unordered set of books (unique books only)



Set contains only **unique elements**.
Order is not important (in **HashSet**).

3 Set Hierarchy



4 Set Implementations

Implementation	Key Characteristics
HashSet	- Uses hash table - No ordering - Allows one null - Best performance for basic operations (O(1))
LinkedHashSet	- Uses hash table + linked list - Maintains insertion order - Allows one null - Slightly slower than HashSet
TreeSet	- Uses Red-Black tree - Stores elements in sorted order - Does not allow null (throws NullPointerException) - O(log n) performance
EnumSet	- Highly optimized for enum types - Uses bit vector - Very fast and memory efficient - No null allowed

5 Example: Using HashSet

```
import java.util.Set;
import java.util.HashSet;

public class SetExample {
    public static void main(String[] args) {
        Set<String> colors = new HashSet<>();

        colors.add("Blue");
        colors.add("Yellow");
        colors.add("Green");
        colors.add("Red");
        colors.add("Yellow"); // duplicate, ignored

        System.out.println("Set: " + colors);
        System.out.println("Contains 'Yellow'? " +
            colors.contains("Yellow"));
    }
}
```

Output (order may vary):

Set: [Red, Blue, Yellow, Green]
Contains 'Yellow'? true

6 Iterating a Set

```
import java.util.*;

public class IterateSet {
    public static void main(String[] args) {
        Set<Integer> set = new HashSet<>();
        set.add(11);
        set.add(22);
        set.add(33);
        set.add(44);

        // 1. Using for-each loop
        System.out.println("Using for-each:");
        for (Integer i : set) {
            System.out.println(i);
        }

        // 2. Using Iterator
        System.out.println("Using Iterator:");
        Iterator<Integer> it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }

        // 3. Using forEach (Java 8+)
        System.out.println("Using forEach:");
        set.forEach(i -> System.out.println(i));
    }
}
```

7 Comparing Set Implementations

Feature	HashSet	LinkedHashSet	TreeSet	EnumSet
Underlying Data Structure	Hash table	Hash table + Linked list	Red-Black tree	Bit vector
Maintains Insertion Order	No	Yes	Sorted (order)	Natural (enum order)
Allows null	Yes (1 null)	Yes (1 null)	No	No
Performance (add, remove, contains)	O(1)	O(1)	O(log n)	O(1)
Iteration order	Unpredictable	Insertion order	Sorted order	Enum order
Use case	General purpose	Need insertion order	Need sorted data	Enum values

8 Common Set Operations

- add(E e)** : Adds an element (if not already present)
- remove(Object o)** : Removes an element
- contains(Object o)** : Checks if an element exists
- size()** : Returns number of elements
- isEmpty()** : Checks if set is empty
- clear()** : Removes all elements

9 Key Takeaways

- ✓ Set stores unique elements (no duplicates).
- ✓ **HashSet** is fastest, no ordering.
- ✓ **LinkedHashSet** maintains insertion order.
- ✓ **TreeSet** keeps elements sorted.
- ✓ **EnumSet** is specialized for enum types.
- ✓ Use Set when you care about uniqueness, not the order (unless required).

“

A Set is like a group of unique items – no repeats, just distinct values.

”

Java PriorityQueue

Based on Priority, Not Insertion Order
(Min-Heap by Default)



Concept | Internal Structure | Common Methods | Example with Custom Object | Iteration | Output

1 What is PriorityQueue?

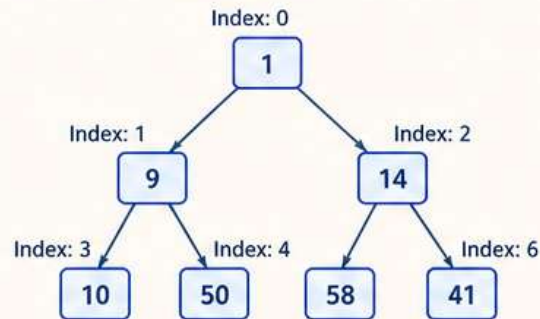
- PriorityQueue is a part of java.util package.
- It stores elements based on their priority.
- By default, it is a **min-heap** (smallest element has highest priority).
- Elements are not stored in insertion order.
- Duplicates are allowed.
- Useful when you always want to process the highest (or lowest) priority element first.



Think of a priority queue like a hospital triage queue — patients with higher priority (more critical) are treated first.

2 Internal Structure (Min-Heap)

PriorityQueue is implemented using a **binary heap** (usually a min-heap) internally.



Internally stored in an array (level order):

1	9	14	10	50	58	41
---	---	----	----	----	----	----

3 Common Methods

Method	Description
<code>add(E e)</code>	Adds an element to the queue.
<code>offer(E e)</code>	Adds an element (returns false if it cannot be added).
<code>peek()</code>	Retrieves, but does not remove, the head element (returns null if empty).
<code>element()</code>	Retrieves, but does not remove, the head element (throws exception if empty).
<code>poll()</code>	Retrieves and removes the head element (returns null if empty).
<code>remove()</code>	Retrieves and removes the head element (throws exception if empty).
<code>size()</code>	Returns the number of elements.
<code>isEmpty()</code>	Checks if the queue is empty.
<code>iterator()</code>	Returns an iterator (elements are not guaranteed to be in priority order).

5 Iterating a PriorityQueue

```
PriorityQueue<Integer> queue =
    new PriorityQueue<>();
queue.add(3);
queue.add(1);
queue.add(2);
```

```
System.out.println("Using Iterator:");
Iterator<Integer> it = queue.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```



Iterator does NOT return elements in priority order. To process in priority order, use `poll()` in a loop.

4 Example with Custom Object

Let's store Employee objects in a PriorityQueue based on salary (lowest salary first).

Employee class

```
class Employee {
    private int id;
    private String name;
    private double salary;

    public Employee(int id, String name,
        double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return "[" + id + ", " + name + ", " +
            salary + "]";
    }
}
```

Using PriorityQueue with Comparator

```
import java.util.*;

public class EmployeePQExample {
    public static void main(String[] args) {
        // PriorityQueue with custom comparator
        PriorityQueue<Employee> pq = new PriorityQueue<>(
            (e1, e2) -> Double.compare(e1.getSalary(), e2.getSalary())
        );

        // Adding employees
        pq.add(new Employee(101, "Amit", 50000));
        pq.add(new Employee(102, "Vijay", 30000));
        pq.add(new Employee(103, "Karan", 70000));
        pq.add(new Employee(104, "Jai", 40000));

        System.out.println("Head (lowest salary): " + pq.peek());

        System.out.println("\nProcessing employees by priority:");
        while (!pq.isEmpty()) {
            System.out.println(pq.poll());
        }
    }
}
```

6 Output

Head (lowest salary): [102, Vijay, 30000]

Processing employees by priority:

```
[102, Vijay, 30000]
[104, Jai, 40000]
[101, Amit, 50000]
[103, Karan, 70000]
```

7 Key Takeaways

- ✓ PriorityQueue orders elements based on priority.
- ✓ By default, it is a min-heap.
- ✓ Use a Comparator for custom objects.
- ✓ `peek()` / `poll()` are commonly used to process highest priority element.
- ✓ Iterator does not guarantee priority order.
- ✓ Useful in real-world scenarios like job scheduling, Dijkstra's algorithm, and task management.

“A PriorityQueue always gives you what's important first!”



Java Generics

Type Safety | Reusability | Compile-Time Checking | Cleaner Code

Write once,
Use with any type!

1 What are Generics?

- Generics were introduced in Java 5 (1.5).
- Generics allow you to write classes, interfaces and methods that can work with different types of data.
- They provide type safety, so you can detect errors at compile time instead of runtime.
- They make code reusable and cleaner.



Generics let you write a single implementation that works with any data type.

2 Before vs After Generics

Before Java 1.5 (Raw Types)

```
List list = new ArrayList();
list.add("Hello");
list.add(22);    // Allowed
String s = (String) list.get(0);
Integer i = (Integer) list.get(1);
// Risk of ClassCastException
```

Java 1.5 and Above (Generics)

```
List<String> list = new ArrayList<String>();
list.add("Hello");
list.add(22);    // Compile-time error
String s = list.get(0); // No casting needed
```

3 Syntax of Generics

Generic Class

```
class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
```

Generic Method

```
public static <T> void printArray(T[] array) {
    for (T item : array) {
        System.out.println(item);
    }
}
```

Generic Interface

```
interface Repository<T> {
    void add(T item);
    T get(int id);
}
```

4 Example: Generic Class

```
class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

public class GenericExample {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.set("Java");
        System.out.println(stringBox.get());

        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        System.out.println(intBox.get());
    }
}
```

Output:

Java
100

5 Issues without Generics

```
public class Demo {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("foo");
        list.add(22);    // allowed

        String s = (String) list.get(0);
        Integer i = (Integer) list.get(1);
    }
}
```

Problem:

- You can add any type of object.
- Need to cast while retrieving.
- May get ClassCastException at runtime.
- No type safety.

6 Polymorphic Behaviour with Generics

You can use a superclass type with subclasses.

```
class Animal { }
class Cat extends Animal { }
class Dog extends Animal { }
```

```
List<Animal> animals = new ArrayList<>(); // OK
animals.add(new Cat());
animals.add(new Dog());
```

```
List<Animal> list = new ArrayList<Cat>(); // Error
// Incompatible types
```



Generics are invariant in Java.
List<Animal> is NOT a List<Cat>.
But you can use wildcards when needed (advanced topic).

7 Benefits of Generics



Type Safety

Detects errors at compile time.



Code Reusability

Write once, use with multiple types.



Fewer Runtime Errors

Eliminates ClassCastException in most cases.



Cleaner Code

No need for explicit type casting.

8 Key Takeaways

- Generics introduced in Java 5 (1.5).
- Provide type safety and compile-time checking.
- Allow reusable and flexible code.
- Prevent runtime ClassCastException.
- Use generics for classes, interfaces and methods.
- Avoid mixing raw types and generics.

“Generics make your code safer, cleaner and more powerful.”

1 What is PECS?

- PECS stands for **Producer Extends** **Consumer Super**.
- It is a guideline to decide when to use `? extends` and `? super` in generics.
- It helps you write flexible, reusable and type-safe code.
- It is especially useful when designing methods that work with collections.

“Use **extends** when you only read (from the collection) and **super** when you only write (to the collection).”

2 The PECS Rule

“ **Producer Extends**
Consumer Super ”

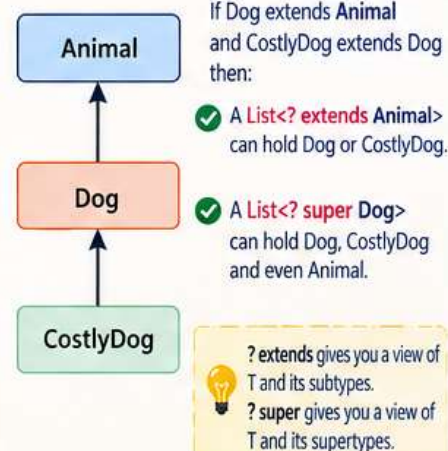
Producer (use extends)

- You only read/get items from the collection.
- Use `? extends T`
- Collection produces `T` (or subtype of `T`)
- You cannot add (except null).

Consumer (use super)

- You only add/put items into the collection.
- Use `? super T`
- Collection consumes `T` (or supertype of `T`)
- You can add `T` (or subtype of `T`). You can read as `Object`.

3 Class Hierarchy Example



4 Using ? extends (Producer)

```
public static void printAnimals(
    List<? extends Animal> animals) {
    for (Animal a : animals) { // OK
        System.out.println(a);
    }
    // animals.add(new Dog()); // Compile error
    // animals.add(new Animal()); // Compile error
    // Can only read, cannot add
}
```

Allowed:

- Read elements as `Animal`
- Actual objects can be `Animal`, `Dog`, `CostlyDog`, etc.

Not Allowed:

- Cannot add (except null)

5 Using ? super (Consumer)

```
public static void addDogs(
    List<? super Dog> list) {
    list.add(new Dog()); // OK
    list.add(new CostlyDog()); // OK
    list.add(new Animal()); // Compile error

    Object obj = list.get(0); // Must read as Object
    // Dog d = list.get(0); // Compile error
}
```

Allowed:

- Add `Dog` or any subtype (e.g. `CostlyDog`)
- Actual collection can be `List<Dog>`, `List<Animal>`, `List<Object>`

Not Allowed:

- Cannot add supertype (e.g. `Animal`)
- When reading, you only get `Object`

6 Examples in Action

A. Read from a list (Producer)

```
static void printList(List<? extends Number> list) {
    for (Number n : list) {
        System.out.println(n);
    }
}

List<Integer> intList = List.of(1, 2, 3);
List<Double> dblList = List.of(1.1, 2.2);
printList(intList); // OK
printList(dblList); // OK
```



Works with any subtype of `Number`.

B. Add to a list (Consumer)

```
static void addNumbers(List<? super Integer> list) {
    list.add(10); // OK
    list.add(20); // OK
    // Integer x = list.get(0); // Compile error
}

List<Integer> intList = new ArrayList<>();
List<Number> numList = new ArrayList<>();
List<Object> objList = new ArrayList<>();

addNumbers(intList); // OK
addNumbers(numList); // OK
addNumbers(objList); // OK
```



Works with `Integer` or any of its supertypes.

7 When to Use What?

Use Case	Wildcard
Method only reads from collection	<code>? extends T</code>
Method only adds to collection	<code>? super T</code>
Method does both (read + add)	No wildcard (use exact type)
You are not sure	Usually no wildcard

8 Key Takeaways

- ✓ PECS = Producer Extends, Consumer Super
- ✓ Use `extends` when you only read from a collection.
- ✓ Use `super` when you only add to a collection.
- ✓ It makes your code more flexible and reusable.
- ✓ Don't use wildcards if you need to both read and write.



Real World Analogy

A box of fruits:

- If you only take fruits out → it can be a box of Apples or a box of any Fruit (**extends**)
- If you only put fruits in → you can safely put Apples into a box of Fruits (**super**)

Remember:

“Use **extends** to get,
use **super** to put!”

Right wildcard.
Stronger code!

1 Generic class

```
class MyObject<T>{
    T myObject;

    public T getMyObject() {
        return myObject;
    }

    public void setMyObject(T myObject) {
        this.myObject = myObject;
    }
}

public class GenClass {

    public static void main(String[] args) {
        MyObject<String> o=new MyObject<String>();
        o.setMyObject(new Integer(22)); //will not compile !!!
    }
}
```

will not compile !!!



Cannot add Integer to MyObject<String>. This will not compile.

2 ? extends XXXXXX (Upper Bound)

```
package com;
import java.util.*;

public class DemoGen1 {
    public static void main(String[] args) {
        List<Integer> list=new ArrayList<Integer>();
        list.add(22);
        list.add(33);

        strangMethod(list);

        for(Integer temp: list)
            System.out.println(temp);
    }

    private static void strangMethod(List<? extends Number> list) {
        list.add(new Integer(22)); //Compile time error..... Good
    }
}
```

in strangMethod() we can pass any derivative of Number class but we are not allowed to modify the list

Upper-bound is when you specify (? extends Field) means argument can be any Field or subclass of Field.

3 ? Super XXXXXX (Lower Bound)

```
class Animal { }

class Cat extends Animal{ }

class Dog extends Animal{ }

class CostlyDog extends Dog{ }

.....

List<Dog>list=new ArrayList<Dog>();
list.add(new Dog("white"));
list.add(new Dog("red"));
list.add(new Dog("black"));
strangMethod(list);

private static void strangMethod(List<? super Dog> list) {
    list.add(new CostlyDog());
}
```

Anytype of Dog is allowed and can also modify list

Lower-bound is when you specify (? super Field) means argument can be any Field or superclass of Field.

4 Generic method

```
class MaxOfThree{

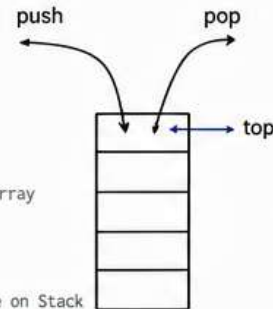
    public static <T extends Comparable<T>> T maxi(T a,T b, T c){
        T max=a;
        if(b.compareTo(a)>0)
            max=b;
        if(c.compareTo(max)>0)
            max=c;
        return max;
    }
}
```

5 Generics – Stack (of Objects) Class

```
class Stack<E>
{
    private final int size;
    private int top;
    private E[] elements;
    public Stack() { this(10); }
    public Stack(int s) {
        size = s > 0 ? s : 10;
        top = -1;
        elements = (E[]) new Object[size]; // create array
    }

    public void push(E pushValue) {
        if (top < size - 1) // if stack is not full
            elements[++top] = pushValue; // place pushValue on Stack
    }

    public E pop() {
        if (top > -1) // if stack is not empty
            return elements[top--]; // remove and return top element of Stack
    }
}
```



6 Generic Dao

